GameCore - Trabalho 3

Como evoluímos o projeto e organizamos a persistência local com Room

Integrantes: Gabriel Henrique Alves Raatz e Matheus Pupia

Repositório: github.com/GabrielRaatz18/GameCorePart3

1. Do Trabalho 2 para o Trabalho 3

No Trabalho 2, nós mantínhamos as avaliações e a Lista de desejos em memória. Conseguíamos adicionar e remover itens, mas perdíamos os dados ao fechar o aplicativo. Nesta etapa, mantivemos as telas e passamos a salvar esses registros no SQLite local com Room. Também criamos cadastro e login usando o mesmo banco.

Decidimos deixar o carrinho e a Biblioteca temporários. Usamos a compra simulada para colocar o jogo na Biblioteca durante a sessão e liberar o formulário de avaliação. Mantivemos o catálogo de jogos no código, sem criar uma tabela para ele nesta etapa.

2. Estrutura do banco

Organizamos o banco em três partes: avaliações, Lista de desejos e cadastro/login. Criamos uma entidade para cada uma:

Entidade / tabela	Campos	Decisão
Avaliações **ReviewData / avaliacoes**	id, gameId, author, rating, text, helpful	Usamos id para identificar cada avaliação e gameId para ligá-la ao jogo. Guardamos também autor, nota, texto e votos úteis.
Lista de desejos **WishlistEntry / lista_desejos**	id, gameId, adicionadoEm	Guardamos o id do jogo e a data da inclusão. Buscamos nome, capa e preço no catálogo para não duplicar esses dados.
Cadastro e login **Usuario / usuarios**	id, nome, usuario, email, senha	Salvamos os dados informados no cadastro. No login, consultamos e-mail e senha para encontrar a conta.

Anotamos as três classes com **@Entity** e usamos **id: Int = 0** com **@PrimaryKey(autoGenerate = true)**. Criamos um índice único em gameId na Lista de desejos para não salvar o mesmo jogo duas vezes.

Reunimos as três entidades e os DAOs em **AppDatabase**. Criamos uma única instância com Room.databaseBuilder, companion object, @Volatile e synchronized. Quando acrescentamos os usuários, passamos o banco para a versão 2 e criamos a migração 1 para 2 sem apagar avaliações nem desejos.

> Criamos cadastro e login como complemento ao Trabalho 3, embora o enunciado não os exigisse. Fizemos tudo localmente, sem servidor ou sincronização. Nesta versão de estudo, guardamos a senha diretamente no banco; sabemos que isso não seria adequado para um aplicativo publicado.

3. DAOs e consultas

Criamos **ReviewDao**, **WishlistDao** e **UsuarioDAO**, um para cada tabela. Nos três, implementamos métodos suspend para inserir (@Insert), buscar todos (@Query), atualizar (@Update) e remover (@Delete). Acrescentamos buscas por id para localizar registros específicos.

Em ReviewDao, usamos buscarPorJogoId(gameId) para mostrar só as avaliações do jogo aberto. Em WishlistDao, usamos a busca por gameId para localizar um item antes de removê-lo. Em UsuarioDAO, consultamos id, e-mail e nome de usuário; verificamos se já existe uma conta antes do cadastro e usamos buscarLogin(email, senha) para entrar.

Executamos os DAOs no GameCoreViewModel com viewModelScope.launch(Dispatchers.IO), fora da thread da interface. Depois, atualizamos o estado do Compose e tratamos erros com try/catch e Log.e.

4. Fluxos nas telas e detalhe

• **Buscar:** ao abrir a Lista de desejos ou Avaliações, carregamos os registros com LaunchedEffect. Filtramos as avaliações pelo jogo.

• **Inserir:** salvamos o jogo pelo botão do detalhe e publicamos avaliações pelo formulário.

• **Atualizar:** no detalhe da avaliação, alteramos nota ou texto e salvamos pelo DAO.

• **Remover:** apagamos desejos e avaliações pelos botões das telas. Não fizemos edição de desejos na interface; esse DAO tem atualizar, mas a tabela guarda só o jogo e a data.

No detalhe da avaliação, enviamos apenas **reviewId** pela navegação. Buscamos o registro pelo id no ReviewDao, em vez de passar o objeto inteiro entre telas.

No **Cadastro**, verificamos se e-mail e usuário já existem antes de gravar a conta. No **Login**, consultamos e-mail e senha no UsuarioDAO antes de abrir a Loja. Conseguimos entrar novamente após fechar o app porque mantemos a conta no banco. Criamos atualizar e deletar no DAO, mas não colocamos essas ações na tela de usuários.

5. Dificuldades e soluções

Nós, Gabriel e Matheus, tivemos dificuldade para organizar o banco. Decidimos separar avaliações, desejos e usuários em três tabelas. Mantivemos o catálogo no código e usamos gameId para relacionar os desejos aos jogos. Para incluir cadastro e login depois, criamos uma migração que preserva os dados anteriores.

Também tivemos dificuldade ao trocar as listas em memória pelo Room: vimos um jogo aparecer na Lista de desejos antes de o banco confirmar a gravação. Passamos a atualizar a tela somente após inserir e buscar os registros. Controlamos inclusões pendentes para evitar toques repetidos e relermos a lista depois da remoção.

Na primeira configuração, enfrentamos um erro de compilação entre Room, Kotlin e KSP. Atualizamos o Room para a versão 2.8.5 e conseguimos compilar novamente.

6. Evidências e apresentação

Nosso vídeo: assistir à gravação do GameCore no YouTube. Mostramos cadastro, navegação, Lista de desejos e uma avaliação salva.

> No trecho final do vídeo, mostramos a avaliação por volta de 1:39, saímos para a tela inicial do celular e abrimos o app de novo. Por volta de 1:59, mostramos a mesma avaliação após a reabertura. Assim, demonstramos que o registro continuou salvo.

Na apresentação em sala, podemos repetir a publicação, a edição e a remoção de uma avaliação e reabrir o aplicativo para mostrar um dado salvo. Não vamos limpar os dados nem desinstalar o app durante o teste, pois isso apagaria o banco local.

7. Como executar

Para executar o projeto, abrimos a pasta no Android Studio, sincronizamos o Gradle e rodamos o módulo app em Android 7.0 (API 24) ou superior. No primeiro acesso, cadastramos uma conta. Para testar avaliações, concluímos uma compra simulada e abrimos o jogo na Biblioteca; para testar a Lista de desejos, usamos o botão no detalhe do jogo.